

FLEETLOGIX

SISTEMA DE GESTIÓN DE TRANSPORTE Y LOGÍSTICA

Proyecto integrador: AVANCE 1

Alumno: DENNYS DURAN

Materia: dspt06

1. DESCRIPCIÓN DEL PROYECTO

FleetLogix es un sistema de gestión logística orientado al control de vehículos, conductores, rutas, viajes, entregas y mantenimientos. El proyecto utiliza una base de datos relacional implementada en SQL Server y un script en Python para generar datos sintéticos coherentes con operaciones reales de transporte.

El objetivo principal fue diseñar un modelo de datos funcional, poblarlo con información realista y validar la calidad de los registros antes de su carga final.

2. MODELO RELACIONAL

La base de datos está compuesta por 6 tablas distribuidas entre entidades maestras y transaccionales.

TABLA	TIPO	REGISTROS	DESCRIPCIÓN
vehicles	Maestra	200	Vehículos de la flota
drivers	Maestra	400	Conductores de la flota
routes	Maestra	50	Rutas predefinidas
trips	Transaccional	100,000	Viajes realizados
deliveries	Transaccional	400,147	Entregas por viajes
maintenance	Transaccional	5,000	Historial de mantenimiento

Las tablas maestras se generan primero para respetar las dependencias de claves foráneas antes de poblar las tablas transaccionales.

Relaciones principales

- Un vehículo puede realizar múltiples viajes.
- Un conductor puede cubrir múltiples viajes.
- Una ruta puede utilizarse en múltiples viajes.
- Un viaje puede contener múltiples entregas.
- Un vehículo puede registrar múltiples mantenimientos.

El modelo fue representado mediante un diagrama ER usando notación Crow's Foot.

3. GENERACIÓN DE DATOS SINTÉTICOS

Se desarrolló un script en Python para generar más de 505 mil registros sintéticos con consistencia referencial y lógica operativa realista.

Herramientas utilizadas

- Python 3.x
- Faker (es_MX)
- pandas
- numpy
- SQLAlchemy + pyodbc
- logging

Orden de carga

- vehicles
- drivers
- routes
- trips
- deliveries
- maintenance

Lógica de negocio implementada

Los datos fueron diseñados para simular operaciones reales de transporte:

- Camiones operan principalmente en horarios nocturnos.
- Vans priorizan entregas urbanas matutinas.
- Motocicletas cubren operaciones de última milla.
- El peso de carga se mantiene dentro de rangos coherentes según la capacidad del vehículo.
- Los viajes se distribuyen históricamente entre 2023 y 2024.

Funciones principales

`generate_trips()`

La función genera viajes asegurando:

- Integridad referencial.
- Fechas consistentes.
- Distribución horaria realista.
- Coherencia entre capacidad y peso transportado.

`get_hourly_distribution()`

Función auxiliar que asigna probabilidades horarias según el tipo de vehículo para simular patrones reales de operación.

4.CONTROL DE CALIDAD

Antes de cargar los datos en SQL Server se ejecutaron validaciones automáticas de calidad.

Validaciones aplicadas

Integridad referencial

Se verificó que todas las claves foráneas existieran en sus respectivas tablas maestras.

Consistencia temporal

- arrival_datetime > departure_datetime
- delivered_datetime > scheduled_datetime
- maintenance_date >= acquisition_date

Resultado

CONTROL DE CALIDAD PASSED — 0 errores encontrados.

Todos los registros fueron validados exitosamente antes de su inserción en SQL Server.

Además, se implementó un sistema de logging para registrar la ejecución completa del proceso de carga.

5.CONTROL DE CALIDAD

El modelo fue adaptado desde PostgreSQL hacia SQL Server 17.0.

Cambios principales

- SERIAL → INT IDENTITY(1,1)
- BOOLEAN → BIT
- pg_indexes → sys.indexes

Índices creados

- idx_trips_departure
- idx_deliveries_status
- idx_vehicles_status

RESULTADO DE CARGA

TABLA	REGISTROS
vehicles	200
drivers	400
routes	50
trips	100,000
deliveries	400,147
maintenance	5,000

Total cargado: 505,797 registros.

La verificación final confirmó que todos los conteos en SQL Server coincidían exactamente con los registros generados en Python.

CONCLUSIONES

El proyecto permitió diseñar e implementar un entorno de datos completo para una operación logística simulada, integrando:

- Modelado relacional.
- Generación masiva de datos sintéticos.
- Validaciones de calidad.
- Carga automatizada en SQL Server.
- Simulación de reglas reales de negocio.

La solución quedó preparada para futuras etapas de análisis, visualización y explotación analítica de datos.